Module 05 — File Handling and Logging

Phase: Advanced Python
Previous module: Exception Handling
Next module: Git and GitHub

Module Outcome:

- File open modes
- `with` statement (context manager)
- Reading strategies
- CSV module
- Logging levels

What is file handling? What is logging?

Programs often need to **remember data after they close** — student records, expense rows, settings. **Files** on disk hold that data. **Logging** writes timestamped messages about what the program did (or failed to do) so you can debug later.

Why learn files and logging?

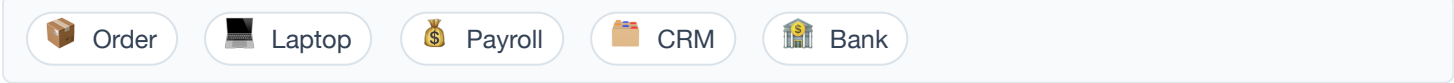
Programs still sit on files (logs, media, CSV import/export, config JSON). If you only know `print()`, you will be blind on a server where nobody watches the console.

Opening a file is like borrowing a library book:

Mode	Plain meaning
`r`	Read only — book unchanged
`w`	Write new — **replaces** old content (dangerous if you meant append)
`a`	Append — add at the end (good for logs)

The `with` statement is the polite borrower — Python **closes** the file even if an error occurs.

CSV = Excel-friendly tables. **JSON** = language of APIs and config. Prefer them over inventing your own text format.



Where used?
- **AcmeHR** / payroll exports — append CSV rows, then summarise by category
- Attendance & logs — append-only text files that survive process restarts
- Config — `prefs.json` / `config.json` loaded at startup
- Server debugging — `logging` to `app.log` when nobody watches the console
- **TechMart** reports — iterate line-by-line or `csv.DictReader` instead of loading everything

Important Concepts

1. File open modes — what / why, then table

Definition: Letter codes that say how Python may use the file.
Purpose: Wrong mode silently wipes data (`w`) or refuses to open (`r` on missing file).

Mode	Meaning	File must exist?	Truncates?
`r`	Read text	Yes	No
`w`	Write text	No (creates)	Yes (clears)
`a`	Append text	No (creates)	No
`r+`	Read + write	Yes	No
`rb`, `wb`	Binary read/write	Varies	Varies

Always specify `encoding="utf-8"` for text on cross-platform projects.

Note:
- `w` mode by mistake deletes existing data — confirm the mode; back up files first.
- Ignoring encoding → `UnicodeDecodeError` on Windows — always use `encoding="utf-8"`.

Common Error: Opening with `"w"` when you meant `"a"` — `w` truncates the file to empty before you write.

Try these (2 programs):

Program 1 — Write / read / truncate demo

```
with open("notes.txt", "w", encoding="utf-8") as f:
    f.write("First draft\n")

with open("notes.txt", "r", encoding="utf-8") as f:
    print("After first write:", repr(f.read()))

with open("notes.txt", "w", encoding="utf-8") as f:
    f.write("Second draft — first line is gone\n")

with open("notes.txt", "r", encoding="utf-8") as f:
    print("After truncate write:", repr(f.read()))
```

Expected Output:

```
After first write: 'First draft\n'
After truncate write: 'Second draft — first line is gone\n'
```

Program 2 — Append attendance lines

```
with open("attendance.txt", "w", encoding="utf-8") as f:
    f.write("Anita present\n")

with open("attendance.txt", "a", encoding="utf-8") as f:
    f.write("Ravi present\n")
    f.write("Meera present\n")
```

```
    f.write("Vikram present\n")

with open("attendance.txt", "r", encoding="utf-8") as f:
    print(f.read())  # all four lines remain
```

Expected Output:

```
Anita present
Ravi present
Meera present
Vikram present
```

2. with statement (context manager)

Definition: A block that acquires a resource and guarantees cleanup.

Purpose: Forgotten `f.close()` leaks handles; `with` closes even after exceptions.

```
with open("notes.txt", "w", encoding="utf-8") as f:
    f.write("Hello KSR batch\n")
# file automatically closed here
```

Note:
- Forgetting to close a file → resource leak — use `with`.

Remember: Prefer `with open(...) as f:` — the file closes even if an exception fires inside the block.

Try these (2 programs):

Program 1 — Rewrite with `with open`

```
# Old style (avoid):
# f = open("batch.txt", "w", encoding="utf-8")
# f.write("PFS Hyderabad\n")
# f.close()

with open("batch.txt", "w", encoding="utf-8") as f:
    f.write("PFS Hyderabad\n")

with open("batch.txt", "r", encoding="utf-8") as f:
    print(f.read().strip())
```

Expected Output:

```
PFS Hyderabad
```

Program 2 — File still closes after exception

```
try:
    with open("scratch.txt", "w", encoding="utf-8") as f:
        f.write("partial data\n")
        raise ValueError("deliberate crash after write")
except ValueError as e:
    print("Caught:", e)
```

```
# File is closed here — safe to reopen
with open("scratch.txt", "a", encoding="utf-8") as f:
    f.write("appended after crash\n")

with open("scratch.txt", "r", encoding="utf-8") as f:
    print(f.read())
```

3. Reading strategies

Method	Returns	Best for
`read()`	Entire file string	Small files
`readline()`	One line	Line protocols
`readlines()`	List of lines	Medium files
`for line in f`	Iterator	**Large files**

Note:
- Reading a huge file with `read()` → `MemoryError` — iterate line by line instead.

Try these (2 programs):

Program 1 — Hyderabad metro routes line-by-line

```
stops = [
    "Miyapur",
    "JNTU College",
    "KPHB Colony",
    "Kukatpally",
    "Balanagar",
]
with open("routes.txt", "w", encoding="utf-8") as f:
    for stop in stops:
        f.write(stop + "\n")

with open("routes.txt", "r", encoding="utf-8") as f:
    for line in f:
        print(line.strip())
```

Expected Output:

```
Miyapur
JNTU College
KPHB Colony
Kukatpally
Balanagar
```

Program 2 — Why loop beats `read()` for huge logs

```
# Comment: f.read() loads the ENTIRE file into RAM.
# A 2 GB server log would need ~2 GB memory at once.
# for line in f: yields one line at a time — safe for huge files.

with open("routes.txt", "w", encoding="utf-8") as f:
    f.write("line1\nline2\nline3\n")
```

```
# Small-file demo of both styles:
with open("routes.txt", "r", encoding="utf-8") as f:
    all_text = f.read()  # OK for small files only
    print("read():", repr(all_text))

with open("routes.txt", "r", encoding="utf-8") as f:
    for line in f:  # prefer this for large logs
        print("line:", line.strip())
```

4. CSV module

```
import csv

with open("students.csv", newline="", encoding="utf-8") as f:
    reader = csv.DictReader(f)
    for row in reader:
        print(row["name"], row["marks"])
```

Note:
- CSV without `newline=""` → extra blank lines — pass `newline=""` on open.

Try these (2 programs):

Program 1 — AcmeHR fees CSV write + total

```
import csv

rows = [
    {"name": "Anita", "amount": "15000"},
    {"name": "Ravi", "amount": "15000"},
    {"name": "Meera", "amount": "12000"},
]

with open("fees.csv", "w", newline="", encoding="utf-8") as f:
    writer = csv.DictWriter(f, fieldnames=["name", "amount"])
    writer.writeheader()
    writer.writerows(rows)

total = 0
with open("fees.csv", newline="", encoding="utf-8") as f:
    for row in csv.DictReader(f):
        total += int(row["amount"])
print("Total fees:", total)  # 42000
```

Expected Output:

```
Total fees: 42000
```

Program 2 — TechMart canteen menu CSV export

```
import csv

menu = [
    {"item": "Samosa", "price": 20},
    {"item": "Idli", "price": 30},
    {"item": "Filter Coffee", "price": 25},
```

```
]

with open("canteen.csv", "w", newline="", encoding="utf-8") as f:
    writer = csv.DictWriter(f, fieldnames=["item", "price"])
    writer.writeheader()
    writer.writerows(menu)

with open("canteen.csv", newline="", encoding="utf-8") as f:
    for row in csv.DictReader(f):
        print(f"{row['item']}: ₹{row['price']}")
```

Expected Output:

```
Samosa: ₹20
Idli: ₹30
Filter Coffee: ₹25
```

5. Logging levels (low → high severity)

Level	Numeric	Use
DEBUG	10	Detailed diagnostic
INFO	20	Normal progress
WARNING	30	Something unexpected
ERROR	40	Failure in function
CRITICAL	50	App may stop

```
import logging
logging.basicConfig(level=logging.INFO, format="%(asctime)s %(levelname)s %(message)s")
logging.info("Server started")
logging.error("DB connection failed")
```

Note:
- Configuring logging in a library at import time → noise — configure logging in `main` only.
- Using `print` in production → no levels/timestamps — use `logging`.

Interview: `print` is for quick demos; `logging` gives levels, timestamps, and file/console handlers you can filter in production.

Try these (2 programs):

Program 1 — INFO vs DEBUG filter

```
import logging

logging.basicConfig(level=logging.INFO, format="%(levelname)s %(message)s")

logging.debug("Parsed raw row — this will NOT appear at INFO level")
```

```
logging.info("Expense added: ₹120 Tea")
logging.error("Failed to parse amount: 'abc'")
```

Expected Output:

```
INFO Expense added: ₹120 Tea
ERROR Failed to parse amount: 'abc'
```

Program 2 — Console + `app.log` handlers

```
import logging

logger = logging.getLogger("expense")
logger.setLevel(logging.DEBUG)
logger.handlers.clear()

fmt = logging.Formatter("%(asctime)s %(levelname)s %(message)s")

file_handler = logging.FileHandler("app.log", encoding="utf-8")
file_handler.setFormatter(fmt)
stream_handler = logging.StreamHandler()
stream_handler.setFormatter(fmt)

logger.addHandler(file_handler)
logger.addHandler(stream_handler)

logger.info("Expense tracker started")
logger.error("Could not open expenses.csv")
print("Also check app.log on disk")
```

Step-by-Step Annotated Python 3 Code

Example 1 — Write and read text file

```
# Write
with open("greeting.txt", "w", encoding="utf-8") as f:
    f.write("Namaste from Python Full Stack batch!\n")
    f.write("Line 2\n")

# Read entire content
with open("greeting.txt", "r", encoding="utf-8") as f:
    content = f.read()
    print(content)

# Read line by line (memory efficient)
with open("greeting.txt", "r", encoding="utf-8") as f:
    for line in f:
        print(line.strip())
```

Example 2 — Append mode for logs

```
from datetime import datetime

def log_visit(username):
    timestamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
```

```
    with open("visits.log", "a", encoding="utf-8") as f:
        f.write(f"{timestamp} | User: {username}\n")

log_visit("ananya")
log_visit("ravi")
```

Example 3 — CSV write and read

```
import csv

# Write CSV
rows = [
    ["roll", "name", "marks"],
    [101, "Meera", 88],
    [102, "Karan", 76],
]

with open("marks.csv", "w", newline="", encoding="utf-8") as f:
    writer = csv.writer(f)
    writer.writerows(rows)

# Read as dict
with open("marks.csv", newline="", encoding="utf-8") as f:
    reader = csv.DictReader(f)
    for row in reader:
        print(f"{row['name']} scored {row['marks']}")
```

Example 4 — JSON config file

```
import json
import os

config = {
    "app_name": "ExpenseTracker",
    "currency": "INR",
    "max_daily_limit": 5000,
    "categories": ["food", "travel", "rent"]
}

config_path = os.path.join("data", "config.json")
os.makedirs("data", exist_ok=True)

with open(config_path, "w", encoding="utf-8") as f:
    json.dump(config, f, indent=2)

with open(config_path, "r", encoding="utf-8") as f:
    loaded = json.load(f)
    print("Daily limit:", loaded["max_daily_limit"])
```

Example 5 — Exception-safe file read

```
def read_text(path):
    try:
        with open(path, "r", encoding="utf-8") as f:
            return f.read()
    except FileNotFoundError:
        print(f"File not found: {path}")
        return None
```

```
    except PermissionError:
        print(f"Permission denied: {path}")
        return None
```

Example 6 — Logging basics

```
import logging

logging.basicConfig(
    level=logging.DEBUG,
    format="%(asctime)s | %(levelname)s | %(name)s | %(message)s",
    datefmt="%Y-%m-%d %H:%M:%S",
    filename="app.log",
    filemode="a"
)

logger = logging.getLogger("expense_app")

def add_expense(amount, category):
    logger.debug("add_expense called amount=%s category=%s", amount, category)
    if amount <= 0:
        logger.warning("Rejected non-positive amount: %s", amount)
        return False
    logger.info("Expense added: ₹%s [%s]", amount, category)
    return True

add_expense(250, "food")
add_expense(-10, "test")
```

Example 7 — Console + file logging

```
logger = logging.getLogger("ksr")
logger.setLevel(logging.DEBUG)
ch = logging.StreamHandler(); ch.setLevel(logging.INFO)
fh = logging.FileHandler("debug.log", encoding="utf-8"); fh.setLevel(logging.DEBUG)
logger.addHandler(ch); logger.addHandler(fh)
logger.info("User login success — console + file")
```

Practice Exercises

Basic

1. Write program that copies `source.txt` to `backup.txt`.
   Model answer:

```
with open("source.txt", encoding="utf-8") as src, open("backup.txt", "w", encoding="utf-8") as dst:
    dst.write(src.read())
```

2. Count lines, words, characters in a text file.
   Model answer:

```
with open("notes.txt", encoding="utf-8") as f:
    text = f.read()
lines = text.splitlines()
```

```
words = text.split()
print(len(lines), len(words), len(text))
```

3. Append current date to `diary.txt` with user note.
   Model answer:

```
from datetime import date

note = input("Note: ")
with open("diary.txt", "a", encoding="utf-8") as f:
    f.write(f"{date.today().isoformat()} | {note}\n")
```

Intermediate

1. Read `marks.csv`, compute class average, write `summary.csv`.
   Model answer:

```
import csv

with open("marks.csv", encoding="utf-8") as f:
    rows = list(csv.DictReader(f))
scores = [float(r["marks"]) for r in rows]
avg = sum(scores) / len(scores) if scores else 0
with open("summary.csv", "w", newline="", encoding="utf-8") as f:
    w = csv.writer(f)
    w.writerow(["count", "average"])
    w.writerow([len(scores), round(avg, 2)])
```

2. Load/save user preferences in `prefs.json`.
   Model answer:

```
import json
from pathlib import Path

path = Path("prefs.json")
prefs = {"theme": "light", "lang": "en"}
if path.exists():
    prefs = json.loads(path.read_text(encoding="utf-8"))
prefs["theme"] = "dark"
path.write_text(json.dumps(prefs, indent=2), encoding="utf-8")
```

3. Configure logger writing INFO+ to file, WARNING+ to console.
   Model answer:

```
import logging

logger = logging.getLogger("app")
logger.setLevel(logging.INFO)
fh = logging.FileHandler("app.log")
fh.setLevel(logging.INFO)
ch = logging.StreamHandler()
ch.setLevel(logging.WARNING)
fmt = logging.Formatter("%(levelname)s %(message)s")
fh.setFormatter(fmt)
ch.setFormatter(fmt)
```

```
    logger.addHandler(fh)
    logger.addHandler(ch)
```

Interview-style

1. Difference between `json.load` and `json.loads`?
   Model answer:
   `json.load` reads from a file-like object. `json.loads` parses a JSON string already in memory (`s` for string).
2. Why `with open` is preferred?
   Model answer:
   The context manager closes the file even if an exception occurs, avoiding resource leaks and ensuring data is flushed. Manual `close()` is easy to skip on error paths.
3. When use CSV vs JSON vs database?
   Model answer:
   CSV suits flat tabular exports and spreadsheets. JSON fits nested config/API payloads. A database is better for concurrent writes, queries, relationships, and integrity constraints in multi-user apps.

Mini-Project (Assignment): AcmeHR Expense Tracker with File + Logging

Title: DailyExpenseLogger
Lab kit (build here): 15_Lab_Assets/01_python_expense_tracker/
Also use: 15_generator_csv_stream — now wire `yield` to `csv.DictReader` (Files-week stretch)

What to build

Piece	Detail
`expenses.csv`	Append: date, category, amount, note
`data/config.json`	Categories + daily limit
`app.log`	All operations via `logging`
Summary	Total by category from CSV
Resilience	Missing file / bad CSV row → clear message, no crash

Folder layout: `main.py`, `data/config.json`, `expenses.csv`, `app.log`

Stretch

- Enforce daily limit from config (warn or block when exceeded).
- Add a “export month report” that writes a new CSV.
- Stream a large seed file with a generator instead of `readlines()`.

Done when

- [ ] Starter kit / your CLI can add expense and show category totals
- [ ] Logging writes to `app.log` (not only `print`)
- [ ] Bad/missing files handled without traceback for expected cases
- [ ] Pushed to GitHub with sample `config.json` (no secrets)

Interview Q&A

Shared with students after concepts and the mini-project. Use for review and interview prep — questions and model answers are included.

1. Why is append mode safer than read-modify-write for log files?
2. Binary vs text mode — when does each matter?
3. What sensitive data should never go in log files (GDPR/India DPDP awareness)?
4. CSV vs JSON for config — trade-offs?
5. How would you rotate log files when they grow too large? (mention `RotatingFileHandler`)

Q1. Difference between `read()` and iterating file object?

A: `read()` loads entire file into memory. Iteration reads one line at a time — constant memory for large files.

Q2. Purpose of logging module vs print?

A: Logging provides levels, timestamps, handlers (file/console), configurable format, and production-ready filtering; print is ad-hoc debugging only.

Q3. What is context manager?

A: Object used with `with` that defines `__enter__` and `__exit__` for setup/cleanup — `open()` closes file in `__exit__`.

Q4. JSON vs CSV?

A: CSV flat tabular, Excel-friendly; JSON nested structures, web API standard. Choose based on consumer.

Q5. How handle missing file?

A: try/except `FileNotFoundError`, or `pathlib.Path.exists()` check; return default or prompt user.
